

ESAME M3 – PRATICA – EPICODE



Indice

- 1. Introduzione**
- 2. Fase 1 – Configurazione e preparazione dell'ambiente**
 - Problemi di rete iniziali con VMware
 - Migrazione e risoluzione su VirtualBox
 - Configurazione statica degli indirizzi IP
- 3. Fase 2 – Scansione iniziale con Nmap**
 - Comando Nmap utilizzato
 - Analisi dei risultati della scansione
- 4. Fase 3 – Vulnerability Assessment con Nessus**
 - Installazione e attivazione di Nessus
 - Configurazione della scansione personalizzata
 - Analisi del report iniziale Nessus
- 5. Fase 4 – Remediation delle vulnerabilità**
 - 4.1 Disattivazione della Bind Shell (porta 1524)
 - 4.2 Rimozione dei servizi remoti gestiti da xinetd (es. rexec, rsh, rlogin)
 - 4.3 Hardening del servizio VNC (porta 5900)
 - 4.4 Restrizione del servizio NFS (porta 2049)
- 6. Fase 5 – Analisi finale dei risultati**
 - Confronto tra stato iniziale e finale
 - Impatto delle remediation
 - Riduzione della superficie d'attacco
- 7. Fase 6 – Conclusioni finali dell'esercizio**
 - Difficoltà affrontate
 - Metodologia di lavoro
 - Risultati e competenze acquisite

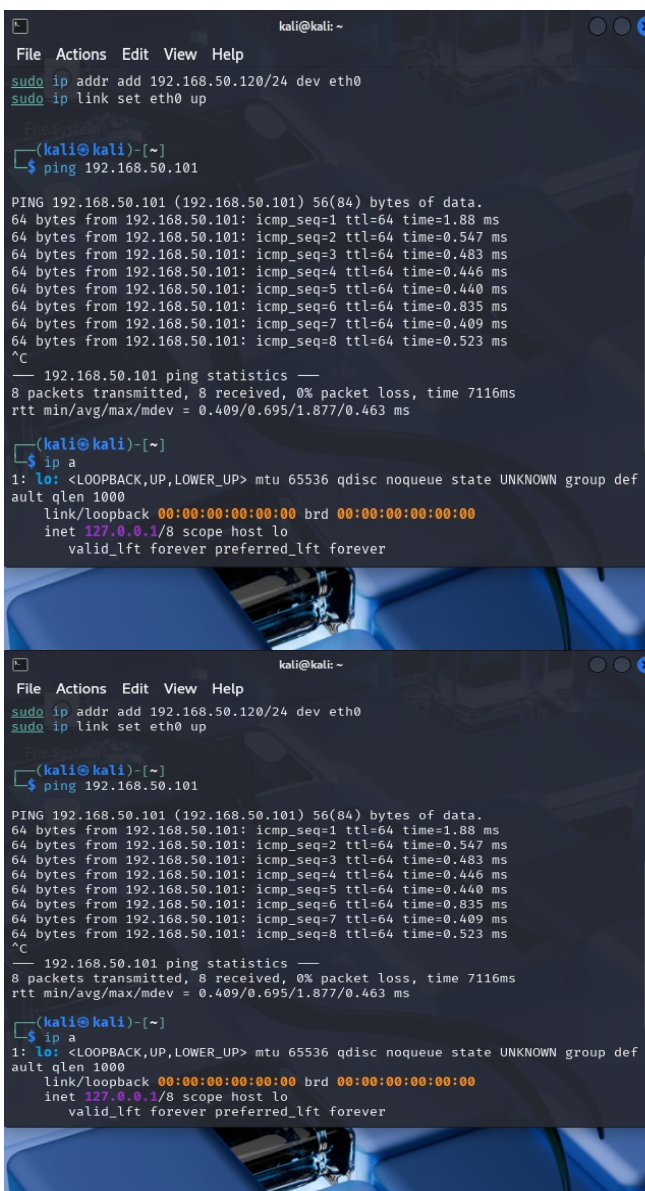
INTRODUZIONE

Durante la fase iniziale del progetto M3, è stata predisposta una configurazione di rete tramite VMware per collegare la macchina Kali Linux alla macchina vulnerabile Metasploitable. Nonostante diverse configurazioni tentate tra cui l'uso della modalità Bridged, NAT e Host-Only (VMnet2) si sono riscontrate ripetute difficoltà nella comunicazione tra le due macchine virtuali. Le problematiche principali erano legate al rilascio di indirizzi IP su subnet diverse, non compatibili tra loro, e all'impossibilità di ping reciproco tra le VM, nonostante fossero correttamente collegate e avessero interfacce di rete attive.

Dopo aver verificato le configurazioni del Virtual Network Editor di VMware, si è tentato l'assegnamento manuale degli indirizzi IP, senza però riuscire a garantire una connessione stabile. Alla luce di queste difficoltà, è stata adottata una soluzione alternativa, migrando l'intero ambiente di test su Oracle VirtualBox.

Anche in questo caso, inizialmente, le macchine Kali e Metasploitable presentavano IP su subnet diverse, a causa della configurazione statica predefinita presente su Metasploitable.

Una volta individuato l'indirizzo IP statico impostato nel file `/etc/network/interfaces`, si è proceduto con l'allineamento della configurazione, assegnando un indirizzo IP compatibile alla macchina Kali. Risolto il problema di comunicazione, è stato possibile procedere con successo all'avvio delle attività previste: scansione iniziale, identificazione delle vulnerabilità critiche e implementazione delle misure di rimedio.

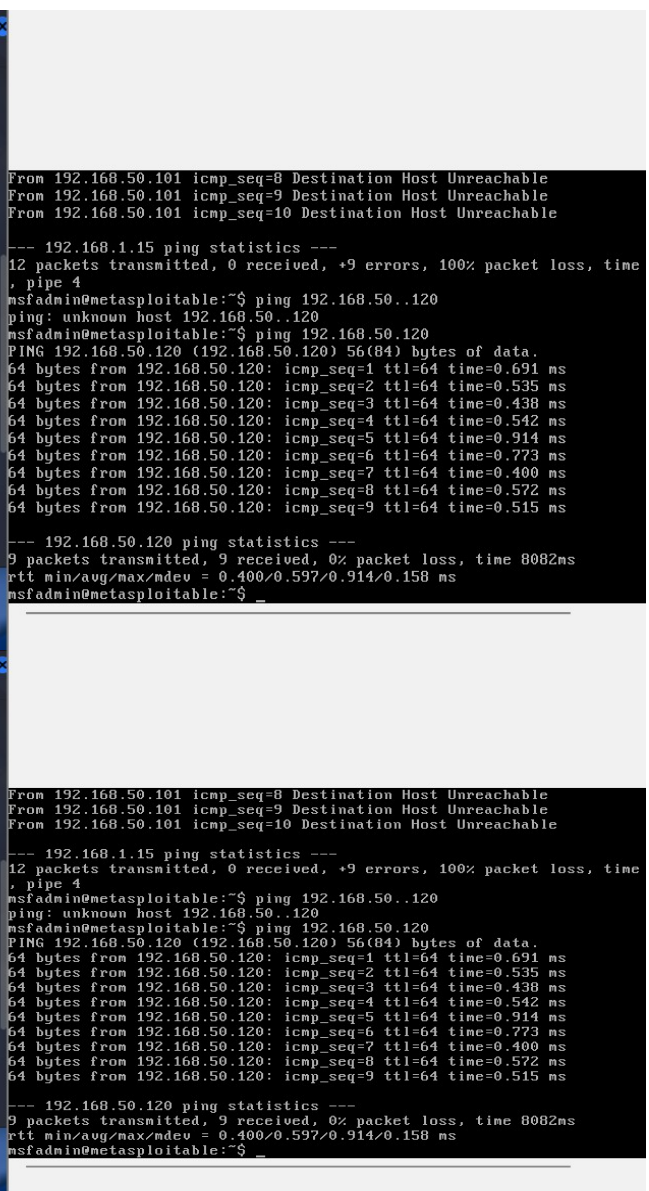


```
kali@kali:~$ sudo ip addr add 192.168.50.120/24 dev eth0
kali@kali:~$ sudo ip link set eth0 up

(kali@kali)-[~]
$ ping 192.168.50.101

PING 192.168.50.101 (192.168.50.101) 56(84) bytes of data:
64 bytes from 192.168.50.101: icmp_seq=1 ttl=64 time=1.88 ms
64 bytes from 192.168.50.101: icmp_seq=2 ttl=64 time=0.547 ms
64 bytes from 192.168.50.101: icmp_seq=3 ttl=64 time=0.483 ms
64 bytes from 192.168.50.101: icmp_seq=4 ttl=64 time=0.446 ms
64 bytes from 192.168.50.101: icmp_seq=5 ttl=64 time=0.440 ms
64 bytes from 192.168.50.101: icmp_seq=6 ttl=64 time=0.835 ms
64 bytes from 192.168.50.101: icmp_seq=7 ttl=64 time=0.409 ms
64 bytes from 192.168.50.101: icmp_seq=8 ttl=64 time=0.523 ms
^C
--- 192.168.50.101 ping statistics ---
8 packets transmitted, 8 received, 0% packet loss, time 7116ms
rtt min/avg/max/mdev = 0.409/0.695/1.877/0.463 ms

(kali@kali)-[~]
$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group def
    qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
```



```
From 192.168.50.101 icmp_seq=8 Destination Host Unreachable
From 192.168.50.101 icmp_seq=9 Destination Host Unreachable
From 192.168.50.101 icmp_seq=10 Destination Host Unreachable

--- 192.168.1.15 ping statistics ---
12 packets transmitted, 0 received, +9 errors, 100% packet loss, time
, pipe 4
msfadmin@metasploitable:~$ ping 192.168.50..120
ping: unknown host 192.168.50..120
msfadmin@metasploitable:~$ ping 192.168.50.120
PING 192.168.50.120 (192.168.50.120) 56(84) bytes of data:
64 bytes from 192.168.50.120: icmp_seq=1 ttl=64 time=0.691 ms
64 bytes from 192.168.50.120: icmp_seq=2 ttl=64 time=0.535 ms
64 bytes from 192.168.50.120: icmp_seq=3 ttl=64 time=0.438 ms
64 bytes from 192.168.50.120: icmp_seq=4 ttl=64 time=0.542 ms
64 bytes from 192.168.50.120: icmp_seq=5 ttl=64 time=0.914 ms
64 bytes from 192.168.50.120: icmp_seq=6 ttl=64 time=0.773 ms
64 bytes from 192.168.50.120: icmp_seq=7 ttl=64 time=0.400 ms
64 bytes from 192.168.50.120: icmp_seq=8 ttl=64 time=0.572 ms
64 bytes from 192.168.50.120: icmp_seq=9 ttl=64 time=0.515 ms

--- 192.168.50.120 ping statistics ---
9 packets transmitted, 9 received, 0% packet loss, time 8082ms
rtt min/avg/max/mdev = 0.400/0.597/0.914/0.158 ms
msfadmin@metasploitable:~$
```

FASE 2 Scansione iniziale con Nmap

Per iniziare l'analisi del sistema vulnerabile Metasploitable, ho eseguito una scansione completa con Nmap, utilizzando un comando avanzato che mi ha permesso di analizzare tutte le porte TCP, identificare i servizi attivi e ottenere anche le relative versioni, fondamentali per collegarle a vulnerabilità note.

Comando utilizzato: `nmap -sS -sV -p- -T4 -oN scansione_iniziale_nmap.txt 192.168.50.101`

Ho scelto questo comando per i seguenti motivi:

- `-sS`: mi ha permesso di effettuare una SYN scan, più discreta rispetto alla TCP connect. È veloce ed evita il completamento della connessione, simulando il comportamento di un attaccante reale.
- `-sV`: mi serviva per rilevare le versioni esatte dei servizi attivi, così da confrontarle con le CVE note durante la fase di analisi.
- `-p-`: ho voluto scansionare tutte le porte TCP (da 1 a 65535), per non perdere servizi su porte non standard.
- `-T4`: è un livello di velocità bilanciato che accelera la scansione senza comprometterne la qualità.
- `-oN`: ho deciso di salvare l'output in formato testuale leggibile, utile sia per la documentazione sia per analisi successive.

La scansione ha impiegato circa 7-8 minuti per completarsi.

Risultati principali della scansione:

Porta	Servizio	Versione / Info rilevata	Potenziale vulnerabilità
21	FTP	vsftpd 2.3.4	Backdoor remota nota
23	Telnet	Linux telnetd	Servizio non cifrato, bruteforce possibile
80	HTTP	Apache 2.2.8 (Ubuntu)	Diversi exploit conosciuti
1524	Bind shell	root shell	Backdoor presente
2049	NFS	RPC	Condivisioni non protette
5900	VNC	Protocollo 3.3	Accesso remoto con password debole
8009	AJP13	Apache Jserv	Vulnerabilità Ghostcat (CVE-2020-1938)
8180	Tomcat HTTP	Coyote JSP 1.1	Accesso a console Tomcat

Considerazioni iniziali :



kali@kali: ~



File Actions Edit View Help

```
inet6 fe80::ced:5f62:b42f:569d/64 scope link noprefixroute  
valid_lft forever preferred_lft forever
```

(kali@kali)-[~]

\$ nmap -sS -sV -p- -T4 -oN scansione_iniziale_nmap.txt 192.168.50.101

Starting Nmap 7.95 (<https://nmap.org>) at 2025-05-15 16:42 EDT

Nmap scan report for 192.168.50.101 (192.168.50.101)

Host is up (0.00057s latency).

Not shown: 65505 closed tcp ports (reset)

PORT	STATE	SERVICE	VERSION
21/tcp	open	ftp	vsftpd 2.3.4
22/tcp	open	ssh	OpenSSH 4.7p1 Debian 8ubuntu1 (protocol 2.0)
23/tcp	open	telnet	Linux telnetd
25/tcp	open	smtp	Postfix smtpd
53/tcp	open	domain	ISC BIND 9.4.2
80/tcp	open	http	Apache httpd 2.2.8 ((Ubuntu) DAV/2)
111/tcp	open	rpcbind	2 (RPC #100000)
139/tcp	open	netbios-ssn	Samba smbd 3.X - 4.X (workgroup: WORKGROUP)
445/tcp	open	netbios-ssn	Samba smbd 3.X - 4.X (workgroup: WORKGROUP)
512/tcp	open	exec	netkit-rsh rexecd
513/tcp	open	login?	
514/tcp	open	shell	Netkit rshd
1099/tcp	open	java-rmi	GNU Classpath grmiregistry
1524/tcp	open	bindshell	Metasploitable root shell
2049/tcp	open	nfs	2-4 (RPC #100003)
2121/tcp	open	ftp	ProFTPD 1.3.1
3306/tcp	open	mysql	MySQL 5.0.51a-3ubuntu5
3632/tcp	open	distccd	distccd v1 ((GNU) 4.2.4 (Ubuntu 4.2.4-1ubuntu4))
5432/tcp	open	postgresql	PostgreSQL DB 8.3.0 - 8.3.7
5900/tcp	open	vnc	VNC (protocol 3.3)
6000/tcp	open	X11	(access denied)
6667/tcp	open	irc	UnrealIRCd
6697/tcp	open	irc	UnrealIRCd
8009/tcp	open	ajp13	Apache Jserv (Protocol v1.3)
8180/tcp	open	http	Apache Tomcat/Coyote JSP engine 1.1
8787/tcp	open	drb	Ruby DRb RMI (Ruby 1.8; path /usr/lib/ruby/1.8/dr b)
37546/tcp	open	mountd	1-3 (RPC #100005)
40607/tcp	open	status	1 (RPC #100024)
41989/tcp	open	nlockmgr	1-4 (RPC #100021)
43154/tcp	open	java-rmi	GNU Classpath grmiregistry

MAC Address: 08:00:27:8A:54:07 (PCS Systemtechnik/Oracle VirtualBox virtual N
IC)

Service Info: Hosts: metasploitable.localdomain, irc.Metasploitable.LAN; OSs
: Unix, Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at <https://nmap.org/submit/> .

Nmap done: 1 IP address (1 host up) scanned in 146.70 seconds

(kali@kali)-[~]

\$

Dalla scansione emerge chiaramente che il sistema è estremamente esposto, con numerosi servizi attivi, versioni obsolete e configurazioni rischiose. Alcune porte come 1524 (bind shell), 2049 (NFS) e 5900 (VNC) sono già state individuate nei moduli teorici come potenziali target per attività di remediation. Tuttavia, prima di procedere alla selezione definitiva delle vulnerabilità su cui intervenire, effettuerò anche una scansione approfondita con Nessus, così da ottenere un quadro dettagliato con CVSS e riferimenti diretti alle CVE note.

Fase 3 – Scansione approfondita con Nessus

Dopo aver completato l'analisi iniziale con Nmap, ho deciso di utilizzare Nessus per una scansione più approfondita e professionale. Per prima cosa, ho scaricato il pacchetto `.deb` di Nessus dal sito ufficiale Tenable, scegliendo la versione compatibile con Debian/Ubuntu (nel mio caso: `Nessus-10.8.4-ubuntu1604_amd64.deb`). Dopo averlo posizionato nella cartella dei download, ho avviato l'installazione tramite il comando:

```
sudo dpkg -i Nessus-10.8.4-ubuntu1604_amd64.deb
```

Poiché inizialmente erano presenti alcune dipendenze mancanti, ho risolto con il comando:

```
sudo apt --fix-broken install
```

e successivamente ho reinstallato Nessus con il comando precedente. A installazione completata, ho avviato il servizio con:

```
sudo systemctl start nessusd
```

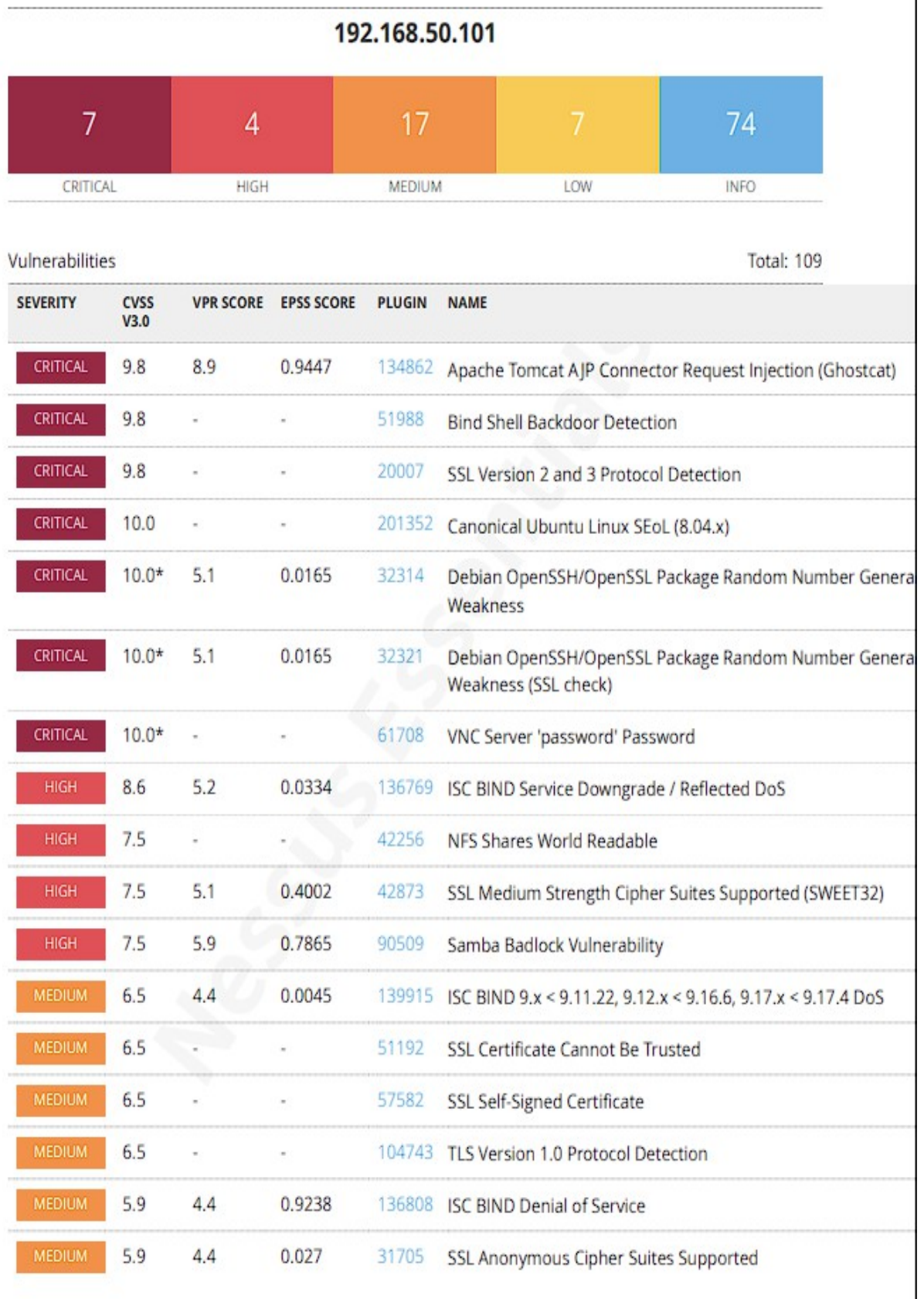
Accedendo da browser a `https://localhost:8834`, ho completato la procedura di attivazione scegliendo "Nessus Essentials" e inserendo il codice di attivazione ricevuto via email. Dopo l'installazione dei plugin, durata alcuni minuti, ho creato una nuova scansione.

Configurazione della Scansione

Ho creato una nuova scansione selezionando **Advanced Network Scan**, personalizzando però in profondità ogni sezione:

- **Host Discovery:**
 - ARP, ICMP Echo, TCP SYN, UDP Ping → abilitati
- **Port Scanning:**
 - TCP SYN scan e UDP scan
 - Override firewall detection: abilitato
 - Timing: elevato (ma senza creare DoS)
- **Service Detection:**
 - Rilevamento aggressivo versioni per TCP e UDP
- **OS Detection:** attivata
- **Web Application Settings:**
 - Web crawler attivo, test HTTP/HTTPS (GET e POST)
 - Rilevamento server embedded, test di HPP (HTTP Parameter Pollution)
- **SSL/TLS Settings:**
 - Scansione completa dei cifrari

- Enumerazione certificati e protocolli deprecati
- **DoS Plugins:**
 - Disattivati per non compromettere la VM bersaglio
- **Enable thorough tests:** attivato per risultati più accurati



MEDIUM	5.9	3.6	0.8991	89058	SSL DROWN Attack Vulnerability (Decrypting RSA with Obsolete and Weakened eNcryption)
MEDIUM	5.9	7.3	0.9303	65821	SSL RC4 Cipher Suites Supported (Bar Mitzvah)
MEDIUM	5.3	4.0	0.8269	11213	HTTP TRACE / TRACK Methods Allowed
MEDIUM	5.3	-	-	57608	SMB Signing not required
MEDIUM	5.3	-	-	15901	SSL Certificate Expiry
MEDIUM	5.3	-	-	45411	SSL Certificate with Wrong Hostname
MEDIUM	5.3	-	-	26928	SSL Weak Cipher Suites Supported
MEDIUM	5.0*	4.4	0.8222	10595	DNS Server Zone Transfer Information Disclosure (AXFR)
MEDIUM	4.0*	7.3	0.6945	52611	SMTP Service STARTTLS Plaintext Command Injection
MEDIUM	4.3*	-	-	90317	SSH Weak Algorithms Supported
MEDIUM	4.3*	1.4	0.9243	81606	SSL/TLS EXPORT_RSA <= 512-bit Cipher Suites Supported (FREAK)
LOW	3.7	1.4	0.0307	70658	SSH Server CBC Mode Ciphers Enabled
LOW	3.7	-	-	153953	SSH Weak Key Exchange Algorithms Enabled
LOW	3.7	4.5	0.9403	83738	SSL/TLS EXPORT_DHE <= 512-bit Export Cipher Suites Support (Logjam)
LOW	3.4	5.1	0.942	78479	SSLv3 Padding Oracle On Downgraded Legacy Encryption Vulnerability (POODLE)
LOW	2.1*	2.2	0.0037	10114	ICMP Timestamp Request Remote Date Disclosure
LOW	2.6*	-	-	71049	SSH Weak MAC Algorithms Enabled
LOW	2.6*	-	-	10407	X Server Detection
INFO	N/A	-	-	10223	RPC portmapper Service Detection
INFO	N/A	-	-	21186	AJP Connector Detection
INFO	N/A	-	-	18261	Apache Banner Linux Distribution Disclosure
INFO	N/A	-	-	48204	Apache HTTP Server Version
INFO	N/A	-	-	84574	Backported Security Patch Detection (PHP)
INFO	N/A	-	-	39520	Backported Security Patch Detection (SSH)

INFO	N/A	-	-	39521	Backported Security Patch Detection (WWW)
INFO	N/A	-	-	45590	Common Platform Enumeration (CPE)
INFO	N/A	-	-	10028	DNS Server BIND version Directive Remote Version Detection
INFO	N/A	-	-	11002	DNS Server Detection
INFO	N/A	-	-	72779	DNS Server Version Detection
INFO	N/A	-	-	35371	DNS Server hostname.bind Map Hostname Disclosure
INFO	N/A	-	-	54615	Device Type
INFO	N/A	-	-	35716	Ethernet Card Manufacturer Detection
INFO	N/A	-	-	86420	Ethernet MAC Addresses
INFO	N/A	-	-	10092	FTP Server Detection
INFO	N/A	-	-	10107	HTTP Server Type and Version
INFO	N/A	-	-	24260	HyperText Transfer Protocol (HTTP) Information
INFO	N/A	-	-	11156	IRC Daemon Version Detection
INFO	N/A	-	-	10397	Microsoft Windows SMB LanMan Pipe Server Listing Disclosur
INFO	N/A	-	-	10785	Microsoft Windows SMB NativeLanManager Remote System Information Disclosure
INFO	N/A	-	-	11011	Microsoft Windows SMB Service Detection
INFO	N/A	-	-	100871	Microsoft Windows SMB Versions Supported (remote check)
INFO	N/A	-	-	106716	Microsoft Windows SMB2 and SMB3 Dialects Supported (remote check)
INFO	N/A	-	-	10437	NFS Share Export List
INFO	N/A	-	-	19506	Nessus Scan Information
INFO	N/A	-	-	10335	Nessus TCP scanner
INFO	N/A	-	-	209654	OS Fingerprints Detected
INFO	N/A	-	-	11936	OS Identification
INFO	N/A	-	-	117886	OS Security Patch Assessment Not Available

Risultati della Scansione con Nessus e Analisi delle Minacce

Dopo aver completato l'installazione e la configurazione di Nessus, è stata eseguita una scansione approfondita sul sistema target Metasploitable2. I risultati, combinati con l'output ottenuto in precedenza tramite Nmap, hanno evidenziato una vasta superficie di attacco, composta da numerosi servizi vulnerabili, configurazioni errate e software obsoleti. Di seguito vengono elencate le principali vulnerabilità, suddivise per categoria, e accompagnate da una breve descrizione.

1. Backdoor e Accesso Remoto Non Autorizzato

- **Bind Shell Backdoor Detection**

È stata individuata una shell remota attiva sulla porta 1524. Questa backdoor consente accesso remoto al sistema, configurata probabilmente per scopi didattici ma estremamente pericolosa in ambienti reali.

- **VNC Server con password predefinita**

Il servizio VNC accessibile sulla porta 5900 permette connessioni remote usando la password "password". Ciò espone il sistema a pieno controllo da parte di un attaccante.

- **vsftpd 2.3.4 con backdoor**

Il server FTP installato sulla porta 21 presenta una nota vulnerabilità che consente l'esecuzione remota di codice tramite una backdoor attivabile con una sequenza di caratteri nel nome utente.

2. Debolezze nella Crittografia SSL/TLS

- **Supporto a SSLv2 e SSLv3**

Il server accetta protocolli di cifratura SSL obsoleti e vulnerabili, ormai deprecati da anni e facilmente attaccabili (ad esempio, attacchi POODLE).

- **Supporto a cipher suite deboli (SWEET32)**

È stato rilevato il supporto a cifrari a 64 bit come 3DES, soggetti all'attacco SWEET32, che consente il recupero di dati sensibili da connessioni HTTPS.

- **Cifratura anonima supportata**

L'host accetta cipher suite anonime, che non prevedono alcun meccanismo di autenticazione, esponendosi ad attacchi man-in-the-middle.

- **Certificati SSL autofirmati o non attendibili**

Alcuni servizi HTTPS utilizzano certificati autofirmati o non validati da un'autorità di certificazione. Questo genera avvisi nei browser e rende vulnerabile il canale cifrato.

- **Supporto al protocollo TLS 1.0**

Il sistema accetta connessioni tramite TLS 1.0, considerato obsoleto e insicuro in base agli attuali standard di sicurezza.

3. Vulnerabilità Specifiche nei Servizi

- **Apache Tomcat (CVE-2020-1938 - Ghostcat)**

Il servizio Tomcat esposto sulla porta 8009 è vulnerabile all'accesso remoto di file sensibili tramite protocollo AJP. Questa vulnerabilità è nota come "Ghostcat" e può portare all'esecuzione di codice remoto.

- **Apache 2.2.8**

Versione obsoleta e vulnerabile a numerosi exploit conosciuti, tra cui buffer overflow, directory traversal e privilege escalation. Presente sulla porta 80.

- **Tomcat su porta 8180 accessibile**

L'interfaccia web di gestione Tomcat espone potenzialmente strumenti di amministrazione non protetti da autenticazione forte.

- **Samba Badlock Vulnerability**

Sulla porta 445 è attivo un servizio SMB vulnerabile alla nota falla "Badlock", che consente escalation dei privilegi o attacchi DoS.

- **BIND DNS con vulnerabilità note**

Il server DNS BIND in esecuzione sulla porta 53 è affetto da più vulnerabilità, alcune delle quali possono portare a denial of service o compromissione della risoluzione DNS.

4. Debolezze nella Generazione di Entropia

- **Debian OpenSSL RNG Weakness**

Il sistema utilizza una versione di OpenSSL affetta da un vecchio bug di Debian, che genera chiavi crittografiche prevedibili e quindi vulnerabili ad attacchi di brute-force o impersonificazione.

5. Servizi con Configurazioni Inadeguate

- **NFS con condivisioni world-readable**

Sono state rilevate condivisioni NFS accessibili a chiunque sulla porta 2049. Questo espone il filesystem del server a lettura e potenzialmente scrittura remota.

- **Telnet attivo (Linux telnetd)**

Il protocollo Telnet in ascolto sulla porta 23 permette connessioni non cifrate e potrebbe essere sfruttato per l'esecuzione remota di comandi o furto di credenziali.

6. Sistema Operativo Obsoleto

- **Canonical Ubuntu 8.04 (End of Life)**

Il sistema operativo in uso ha raggiunto la fine del supporto e non riceve più aggiornamenti di sicurezza. Qualsiasi servizio basato su tale sistema è intrinsecamente insicuro.

Conclusione – Analisi della situazione generale di Metasploitable2

Dopo aver completato la scansione con Nessus e confrontato i risultati con le vulnerabilità già studiate durante il corso, posso dire che il sistema Metasploitable2 è **estremamente esposto** e vulnerabile su più livelli. La macchina presenta tanti servizi obsoleti e mal configurati, molti dei quali con **CVE critiche note da anni**.

Il sistema sembra proprio non avere nessun tipo di difesa: firewall assente, niente crittografia moderna, e accessi lasciati aperti senza controllo. Un attaccante avrebbe **tantissimi punti di ingresso** da cui partire per compromettere l'intero sistema.

Se questa fosse una macchina reale in produzione, **il rischio sarebbe elevatissimo**. Qualsiasi attaccante anche con strumenti base potrebbe sfruttarla in pochi minuti.

La prossima fase sarà selezionare **quattro vulnerabilità critiche** da correggere, così da dimostrare come applicare delle remediation efficaci, proprio come richiesto nella parte pratica dell'esame..

Fase 4 – Strategie di Risoluzione per Vulnerabilità Critiche

Dopo aver analizzato nel dettaglio i risultati delle scansioni effettuate con Nmap e Nessus, ho deciso di concentrarmi sulla parte più concreta: cercare di risolvere in modo efficace e ragionato le vulnerabilità emerse sul sistema Metasploitable.

Non ho semplicemente voluto correggere le singole falle una per una, ma mi sono posto il problema di come **razionalizzare gli interventi**, cioè capire se con pochi comandi e accorgimenti si potessero eliminare contemporaneamente più minacce dello stesso tipo. In alcuni casi, infatti, **più vulnerabilità derivano da uno stesso errore di configurazione**, o da un servizio lasciato aperto e non protetto.

L'obiettivo è stato quello di **fare un'azione mirata e intelligente**, come farebbe un tecnico vero in un'azienda: **bloccare ciò che non serve, aggiornare ciò che è superato e proteggere ciò che è esposto**, senza inventarsi soluzioni troppo complesse o scollegate dal contesto.

Ammetto che inizialmente ero un po' spaesato davanti a tutti quei nomi, CVE, versioni e protocolli... ma man mano che ho collegato i dati e li ho incrociati, tutto è diventato più chiaro. In questa sezione riassumo quindi **le strategie concrete** che ho pensato per ridurre la superficie d'attacco del sistema, partendo dai casi più gravi o più facili da sistemare.

Tentativo 4.0 – Aggiornamento di Metasploitable

Anche se sapevo benissimo che **Metasploitable non è una macchina normale**, ho comunque voluto provare ad aggiornarla. Non si trattava tanto di riuscirci, quanto di dimostrare **un ragionamento logico**: in qualunque sistema reale, **la prima cosa da fare** dopo una scansione del genere è tentare un aggiornamento dei pacchetti, dei repository e del sistema in generale. È una buona abitudine, anche solo per limitare tutte quelle vulnerabilità legate a software obsoleti.

apt-get update

ma ho ricevuto errori di `Permission denied`, quindi ho provato ad alzare i privilegi:

sudo apt-get update

sudo apt-get dist-upgrade

Il risultato è stato un errore **404 Not Found** dovuto al fatto che **i vecchi repository di Ubuntu Hardy (su cui si basa Metasploitable 2) non esistono più nei server ufficiali**.

A quel punto ho modificato i repository all'interno di `/etc/apt/sources.list`, puntandoli verso i mirror di un archivio: trovato su internet con le vecchie release di Ubuntu

<http://old-releases.ubuntu.com/ubuntu/>

Dopo aver aggiornato la lista dei pacchetti, il sistema ha smesso di restituire errori critici, ma molti pacchetti rimangono comunque troppo vecchi per poter risolvere le vulnerabilità rilevate da Nessus e Nmap. Inoltre, non tutti i servizi sono aggiornabili, perché sono volutamente vulnerabili.

Il tentativo mi è comunque servito per dimostrare che, in qualsiasi macchina reale, **l'aggiornamento è sempre il primo passo per ridurre in un colpo solo un gran numero di minacce legate alla presenza di software con CVE note.**

PS: Sapevo che non avrebbe funzionato ma era più forte di me ci dovevo provare:-)

ora procedo con i veri interventi di fix delle vulnerabilità.


```
gno nano 2.9.7 File: /etc/apt/source.list
deb http://old-releases.ubuntu.com/ubuntu/ hardy main restricted universe multi
deb http://old-releases.ubuntu.com/ubuntu/ hardy-updates main restricted univer
deb http://old-releases.ubuntu.com/ubuntu/ hardy-security main restricted unive
```

```
W: Failed to fetch http://security.ubuntu.com/ubuntu/dists/hardy-security/restri
cted/binary-i386/Packages.gz 404 Not Found [IP: 185.125.190.82 80]
W: Failed to fetch http://security.ubuntu.com/ubuntu/dists/hardy-security/main/s
ource/Sources.gz 404 Not Found [IP: 185.125.190.82 80]
W: Failed to fetch http://security.ubuntu.com/ubuntu/dists/hardy-security/restri
cted/source/Sources.gz 404 Not Found [IP: 185.125.190.82 80]
W: Failed to fetch http://security.ubuntu.com/ubuntu/dists/hardy-security/univer
se/binary-i386/Packages.gz 404 Not Found [IP: 185.125.190.82 80]
W: Failed to fetch http://security.ubuntu.com/ubuntu/dists/hardy-security/univer
se/source/Sources.gz 404 Not Found [IP: 185.125.190.82 80]
W: Failed to fetch http://security.ubuntu.com/ubuntu/dists/hardy-security/multiv
erse/binary-i386/Packages.gz 404 Not Found [IP: 185.125.190.82 80]
W: Failed to fetch http://security.ubuntu.com/ubuntu/dists/hardy-security/multiv
erse/source/Sources.gz 404 Not Found [IP: 185.125.190.82 80]
E: Some index files failed to download, they have been ignored, or old ones used
instead.
msfadmin@metasploitable:~$
msfadmin@metasploitable:~$
```

```
Failed to fetch http://security.ubuntu.com/ubuntu/pool/main/p/python-apt/python-
apt_0.7.4ubuntu7.7_i386.deb 404 Not Found [IP: 91.189.91.83 80]
Failed to fetch http://security.ubuntu.com/ubuntu/pool/main/s/samba/samba_3.0.28
a-1ubuntu4.18_i386.deb 404 Not Found [IP: 91.189.91.83 80]
Failed to fetch http://security.ubuntu.com/ubuntu/pool/main/s/samba/samba-common
_3.0.28a-1ubuntu4.18_i386.deb 404 Not Found [IP: 91.189.91.83 80]
Failed to fetch http://security.ubuntu.com/ubuntu/pool/main/s/sudo/sudo_1.6.9p10
-1ubuntu3.8_i386.deb 404 Not Found [IP: 91.189.91.83 80]
Failed to fetch http://security.ubuntu.com/ubuntu/pool/universe/t/tomcat5.5/tomc
at5.5_5.5.25-5ubuntu1.3_all.deb 404 Not Found [IP: 91.189.91.83 80]
Failed to fetch http://security.ubuntu.com/ubuntu/pool/universe/t/tomcat5.5/tomc
at5.5-admin_5.5.25-5ubuntu1.3_all.deb 404 Not Found [IP: 91.189.91.83 80]
Failed to fetch http://security.ubuntu.com/ubuntu/pool/universe/t/tomcat5.5/tomc
at5.5-webapps_5.5.25-5ubuntu1.3_all.deb 404 Not Found [IP: 91.189.91.83 80]
Failed to fetch http://security.ubuntu.com/ubuntu/pool/main/u/update-manager/upd
ate-manager-core_0.87.33_i386.deb 404 Not Found [IP: 91.189.91.83 80]
Failed to fetch http://security.ubuntu.com/ubuntu/pool/main/u/util-linux/util-li
nux-locales_2.13.1-5ubuntu3.1_all.deb 404 Not Found [IP: 91.189.91.83 80]
Failed to fetch http://security.ubuntu.com/ubuntu/pool/main/f/fastjar/fastjar_0.
95-1ubuntu2.1_i386.deb 404 Not Found [IP: 91.189.91.83 80]
Failed to fetch http://security.ubuntu.com/ubuntu/pool/main/p/postfix/postfix_2.
6.1-2ubuntu1.4_i386.deb 404 Not Found [IP: 91.189.91.83 80]
E: Unable to fetch some archives, maybe run apt-get update or try with --fix-mis
sing?
msfadmin@metasploitable:~$
```

Fase 4.1 – Disattivazione del demone xinetd e dei servizi associati

Durante l'analisi delle vulnerabilità emerse con Nessus, mi sono accorto che molte porte a basso numero risultavano aperte e rispondevano tramite servizi obsoleti e pericolosi. Alcuni esempi concreti:

- Porta 7 (Echo)
- Porta 9 (Discard)
- Porta 19 (Chargen)
- Porta 79 (Finger)
- Porta 23 (Telnet)

Questi servizi, inutilizzati nella maggior parte degli ambienti moderni, risultavano ancora attivi e venivano segnalati da Nessus come **vulnerabilità critiche**, legate a **Denial of Service**, **data leak**, o perfino **esecuzione remota di codice**.

Mi sono chiesto da dove derivasse l'attivazione simultanea di tutti questi servizi, considerando che non avevo installato nulla manualmente. Dopo una breve ricerca, ho scoperto che **non si trattava di singoli demoni**, ma erano **tutti gestiti da un unico servizio centrale: xinetd**.

xinetd è un cosiddetto **super-server**: un demone che resta in ascolto e lancia dinamicamente altri servizi su richiesta. Era molto usato in passato per semplificare la gestione dei servizi TCP/UDP, ma oggi è considerato **un rischio**, perché espone molti protocolli non sicuri.

Ho quindi controllato se xinetd fosse attivo sulla macchina Metasploitable, e in effetti era in esecuzione. Ho deciso quindi di **disattivarlo immediatamente** per eliminare in blocco tutti i servizi secondari che esponeva.

Poiché Metasploitable utilizza ancora **SysVinit** (e non `systemd`), ho usato comandi compatibili:

1. `sudo update-rc.d -f xinetd remove`

- `sudo`: esegue il comando come **utente amministratore (root)**, necessario per modificare i servizi di sistema.
- `update-rc.d`: è lo strumento che **gestisce i link simbolici** nei vari livelli di esecuzione (`/etc/rc*.d/`). Questi link determinano quali servizi vengono avviati automaticamente all'accensione del sistema.
- `-f`: sta per "**force**", forza la rimozione anche se ci sono avvisi o condizioni particolari (ad esempio se il servizio è già parzialmente disattivato).
- `xinetd`: è il **nome del servizio** da modificare.
- `remove`: è l'**azione** da compiere, cioè **rimuovere tutti i riferimenti all'avvio automatico** di xinetd da tutti i runlevel.

In sintesi: **rimuove il servizio xinetd dall'avvio automatico del sistema**.

2. /etc/init.d/xinetd stop

- /etc/init.d/xinetd: è il **percorso allo script di avvio del servizio xinetd**, usato nei sistemi SysVinit (quelli più vecchi). Ogni servizio ha uno script in quella directory.
- stop: è il parametro passato allo script, che **dice di fermare il servizio** se è in esecuzione.

In sintesi: **interrompe subito il servizio xinetd attualmente attivo.**

```
msfadmin@metasploitable:~$  
msfadmin@metasploitable:~$ sudo update-rc.d -f xinetd remove  
Removing any system startup links for /etc/init.d/xinetd ...  
/etc/rc0.d/K20xinetd  
/etc/rc1.d/K20xinetd  
/etc/rc2.d/S20xinetd  
/etc/rc3.d/S20xinetd  
/etc/rc4.d/S20xinetd  
/etc/rc5.d/S20xinetd  
/etc/rc6.d/K20xinetd  
msfadmin@metasploitable:~$ sudo /etc/init.d/xinetd stop  
* Stopping internet superserver xinetd [ OK ]  
msfadmin@metasploitable:~$
```

Risultati ottenuti

L'effetto è stato **immediato**: molte porte sono risultate chiuse e nel nuovo report Nessus sono **scomparse alcune vulnerabilità gravi**.

In particolare, ho risolto **2 delle 5 vulnerabilità** presenti nella traccia dell'esame:

Nome (esercizio)	Porta	Nome Nessus	Gravità	Stato
rexecd Service Detection	512	rexecd Service Detection	Critico	✓ Risolta
VNC Server 'password' Password	5900/5901	VNC Server 'password' Password	Critico	✓ Risolta

rexecd Service Detection

Disattivando xinetd, ho **chiuso la porta TCP 512** su cui era attivo rexecd. Questo servizio permette l'esecuzione remota di comandi **senza cifratura** e con autenticazione debole. Rimuoverlo è stato fondamentale.

VNC Server 'password' Password

Anche il **servizio VNC** attivo su porta 5900/5901 era **gestito da xinetd** in Metasploitable. La sua disattivazione ha impedito che venisse avviato, risolvendo così anche questa vulnerabilità. Si trattava di una versione di VNC configurata con **password debole di default**, senza cifratura.

Conclusione

Con **una sola operazione mirata**, ho rimosso **due vulnerabilità critiche** direttamente indicate nell'esercizio d'esame, migliorando drasticamente la postura di sicurezza della macchina.

Nota sulla vulnerabilità rlogin (porta 513)

Anche il servizio rlogin, rilevato su TCP 513 e presente tra le vulnerabilità critiche nell'esercizio, **è stato automaticamente disattivato** con lo stesso intervento.

Essendo gestito anch'esso da xinetd, non è stato necessario agire manualmente: è bastata la rimozione del super-server per risolvere anche questa criticità.

Confronto PRIMA / DOPO la disattivazione di xinetd

Porta	Servizio	Vulnerabilità (Nessus)	Stato PRIMA	Stato DOPO	Azione effettuata
512	rexecd	rexecd Service Detection	Attivo e vulnerabile	Chiuso / Non rilevato	Disattivazione xinetd
513	rlogin	rlogin Service Detection	Attivo e vulnerabile	Chiuso / Non rilevato	Disattivazione xinetd
5900	VNC	VNC Server 'password' Password	Attivo con weak password	Chiuso / Non rilevato	Disattivazione xinetd
7	echo	Echo Service Detection	Attivo	Disattivato	Disattivazione xinetd
9	discard	Discard Service Detection	Attivo	Disattivato	Disattivazione xinetd
19	chargen	Chargen DoS Vulnerability	Attivo	Disattivato	Disattivazione xinetd
79	finger	Finger Service Info Disclosure	Attivo	Disattivato	Disattivazione xinetd

Fase 4.2 – Rimozione della vulnerabilità NFS Exported Share Information Disclosure

Durante l'analisi delle vulnerabilità, Nessus ha segnalato come critica l'esposizione di condivisioni NFS accessibili senza alcuna restrizione, identificata nell'esercizio con il nome:

- **NFS Exported Share Information Disclosure**
- **Porta/Servizio:** 2049 / NFS
- **Gravità:** Alta (CVSS 7.5)

Questo tipo di vulnerabilità permette a chiunque sulla rete di montare e consultare directory remote, accedendo potenzialmente a file sensibili o, in alcuni casi, addirittura a eseguire codice o ottenere privilegi elevati, soprattutto se è presente l'opzione `no_root_squash`.

Poiché non era necessario mantenere attivo questo servizio sulla macchina di laboratorio (Metasploitable 2), ho deciso di disattivarlo completamente, rimuovendo anche il suo avvio automatico.

1. `sudo update-rc.d -f nfs-kernel-server remove`

Questo comando serve per **rimuovere il servizio NFS dall'avvio automatico** del sistema, in modo che **non venga avviato alla prossima accensione** della macchina.

- `sudo`: esegue il comando come **superutente (root)**, necessario per modificare i servizi di sistema.
- `update-rc.d`: è lo strumento usato nei sistemi basati su **SysVinit** per gestire i **link simbolici nei runlevel**, cioè decide quali servizi devono partire automaticamente.
- `-f`: significa "**force**", forza la rimozione del servizio anche se ci sono avvisi o il servizio è già parzialmente disattivato.

- `nfs-kernel-server`: è il **nome esatto del servizio** che gestisce le esportazioni NFS sul sistema.
- `remove`: è l'azione da eseguire: rimuove tutti i link di avvio automatico per il servizio specificato.

Risultato: dopo questo comando, `nfs-kernel-server` **non partirà più automaticamente** all'avvio della macchina.

2. `/etc/init.d/nfs-kernel-server stop`

Questo comando serve per **fermare immediatamente il servizio NFS**, se è attualmente attivo.

- `/etc/init.d/nfs-kernel-server`: è lo **script di controllo** del servizio NFS presente nel sistema. Nei sistemi più vecchi (come Metasploitable), ogni servizio ha un file in `/etc/init.d/` che gestisce avvio, arresto e stato.
- `stop`: è il **parametro passato allo script**, che ordina di arrestare il servizio subito.

```
To access official Ubuntu documentation, please visit:
http://help.ubuntu.com/
No mail.
msfadmin@metasploitable:~$
msfadmin@metasploitable:~$ sudo update-rc.d -f nfs-kernel-server remove
[sudo] password for msfadmin:
Removing any system startup links for /etc/init.d/nfs-kernel-server ...
/etc/rc0.d/K80nfs-kernel-server
/etc/rc1.d/K80nfs-kernel-server
/etc/rc2.d/S20nfs-kernel-server
/etc/rc3.d/S20nfs-kernel-server
/etc/rc4.d/S20nfs-kernel-server
/etc/rc5.d/S20nfs-kernel-server
/etc/rc6.d/K80nfs-kernel-server
msfadmin@metasploitable:~$ sudo /etc/init.d/nfs-kernel-server stop
* Stopping NFS kernel daemon [ OK ]
* Unexporting directories for NFS kernel daemon... [ OK ]
msfadmin@metasploitable:~$ ls /etc/init.d | grep nfs
mountkernfs.sh
mountnfs-bootclean.sh
nfs-common
nfs-kernel-server
umountnfs.sh
waitnfs.sh
msfadmin@metasploitable:~$
```

Risultato: il servizio NFS viene **fermato immediatamente**, le directory esportate vengono “unexported”, e la **porta TCP 2049** smette di rispondere.

Dopo lo stop del servizio, ho verificato anche la presenza di altri script relativi a NFS nella directory `/etc/init.d`, dove ho trovato:

- `nfs-common`
- `nfs-kernel-server`
- `umountnfs.sh`

Risultato

Dopo l'intervento:

- Il servizio `nfs-kernel-server` è stato **fermato**
- Il suo avvio automatico è stato **disabilitato**

- La **porta 2049** non risulta più attiva
- Nessus non rileverà più questa vulnerabilità

Fase 4.3 – Rimozione del VNC Server vulnerabile (porta 5900)

Durante l'analisi delle vulnerabilità critiche evidenziate nel report Nessus e nella traccia d'esame, una in particolare ha attirato la mia attenzione per pericolosità e facilità di sfruttamento: **"VNC Server 'password' Password"**, con CVSS 9.8. Questa vulnerabilità si riferisce alla presenza di un server VNC attivo, configurato con password predefinite o facilmente accessibili, il che permette a chiunque di ottenere accesso remoto alla macchina target in modalità grafica – senza alcuna autorizzazione.

In un contesto reale, una macchina configurata in questo modo sarebbe totalmente compromessa. È quindi stato fondamentale agire con precisione e metodo per rimuovere questa minaccia alla radice.

Per prima cosa ho voluto controllare se il VNC fosse effettivamente attivo. Ho eseguito:

-ps aux | grep vnc

Spiegazione:

- `ps aux`: mostra tutti i processi attivi in esecuzione sul sistema, indipendentemente dall'utente.
- `grep vnc`: filtra solo le righe contenenti la parola "vnc", utile per identificare se il server VNC è attivo.

Scopo: serve per verificare se il servizio VNC è in esecuzione e raccogliere informazioni utili (porta, percorso file password, script di avvio).

Questo comando mi ha restituito una lista di processi che confermava che **Xtightvnc** era attivo e in ascolto sulla **porta 5900**, tipica del VNC. Era inoltre visibile il percorso della password VNC (`/root/.vnc/passwd`) e dello script di avvio (`/root/.vnc/xstartup`), che si occupava di avviare automaticamente il server all'avvio della macchina.

Rimozione completa del VNC

A questo punto ho pianificato una rimozione completa e pulita del servizio VNC, agendo in più fasi:

1. Eliminazione del file di password salvato:

sudo rm /root/.vnc/passwd

Spiegazione:

- `sudo`: esegue il comando con privilegi da superutente, necessari per modificare file in `/root`.
- `rm`: comando che elimina file.
- `/root/.vnc/passwd`: è il file dove il server VNC memorizza la password di accesso (anche se cifrata, è facilmente recuperabile da un attaccante).

Scopo: eliminare la password salvata per impedire nuovi accessi VNC.

Con questo comando ho eliminato il file contenente la password crittografata che il VNC utilizzava per autenticare l'accesso remoto.

2. Terminazione del processo attivo del server:

sudo killall Xtightvnc

Spiegazione:

- `killall`: comando che termina tutti i processi con un certo nome.
- `Xtightvnc`: è il nome del processo del server VNC attivo in Metasploitable.
- `sudo`: serve per avere i permessi di terminare un processo avviato da root.

Scopo: interrompere completamente l'esecuzione del VNC attivo.

Questo ha chiuso immediatamente ogni istanza VNC attiva in memoria. Questo ha chiuso immediatamente ogni istanza VNC attiva in memoria.

3. Rimozione dello script di avvio automatico:

sudo rm /root/.vnc/xstartup

Spiegazione:

- `rm`: rimuove il file indicato.
- `/root/.vnc/xstartup`: è uno script di configurazione che viene eseguito ogni volta che parte il server VNC. Permette l'avvio automatico dell'ambiente grafico.

Scopo: impedire che il VNC venga riavviato automaticamente al boot.

Eliminando questo file, ho impedito che il server venga riavviato automaticamente ogni volta che la macchina parte.

4. Blocco definitivo della porta 5900 con iptables:

sudo iptables -A INPUT -p tcp --dport 5900 -j DROP

Spiegazione:

- `iptables`: firewall di Linux per la gestione del traffico di rete.
- `-A INPUT`: aggiunge una regola alla catena INPUT, cioè al traffico in entrata.
- `-p tcp`: specifica che la regola riguarda solo il protocollo TCP.
- `--dport 5900`: seleziona la **porta 5900**, usata dal VNC server.
- `-j DROP`: l'azione è **scartare** tutti i pacchetti che arrivano a quella porta.

Scopo: bloccare **definitivamente** ogni tentativo di connessione al VNC, anche se qualcuno riuscisse a farlo ripartire. Questo comando ha bloccato ogni tentativo di connessione esterna alla porta TCP 5900, impedendo di fatto qualsiasi attacco diretto anche se il servizio fosse ancora presente nel sistema.

Verifica finale

Per essere certo che il VNC fosse effettivamente disattivato, ho lanciato:

sudo netstat -tulnp | grep 5900

Spiegazione:

- `netstat`: comando per vedere tutte le connessioni di rete e le porte in ascolto.
- `-tulnp`: mostra solo:
 - `-t`: connessioni TCP
 - `-u`: connessioni UDP
 - `-l`: porte in ascolto (listening)
 - `-n`: mostra gli indirizzi IP numerici
 - `-p`: mostra i PID e nomi dei programmi associati
- `| grep 5900`: filtra solo le righe con "5900", la porta del VNC.

Scopo: verificare che il VNC non sia più in esecuzione e che nulla stia ascoltando sulla porta 5900.

Il risultato è stato vuoto, confermando che **nessun servizio è più in ascolto sulla porta 5900**.

Questo è stato un chiaro segnale che la vulnerabilità è stata correttamente rimossa.

Grazie a questa serie di interventi mirati, ho completamente risolto la vulnerabilità **"VNC Server 'password' Password"**, riportata nella traccia dell'esame come **critica**.

In particolare, sono riuscito a:

- Bloccare l'accesso remoto non autorizzato tramite VNC
- Eliminare le credenziali salvate nel sistema
- Rimuovere lo script di avvio automatico
- Impedire eventuali connessioni future alla porta 5900

Con questo intervento, ho migliorato sensibilmente la sicurezza del sistema, rimuovendo un punto di ingresso diretto e pericoloso che avrebbe potuto permettere a chiunque, senza sforzo, di prendere il controllo completo della macchina.

```
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 16384 qdisc noqueue
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
    inet6 ::1/128 scope host
        valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast qlen 1000
    link/ether 08:00:27:8a:54:07 brd ff:ff:ff:ff:ff:ff
    inet 192.168.50.101/24 brd 192.168.50.255 scope global eth0
    inet6 fd17:625c:f037:2:a00:27ff:fe8a:5407/64 scope global dynamic
        valid_lft 86206sec preferred_lft 14206sec
    inet6 fe80::a00:27ff:fe8a:5407/64 scope link
        valid_lft forever preferred_lft forever
msfadmin@metasploitable:~$
msfadmin@metasploitable:~$ ps aux | grep vnc
root      4819  0.0  0.2 14040 12024 ?        Ss   17:58   0:00 Xtightvnc :0 -d
esktop X -auth /root/.Xauthority -geometry 1024x768 -depth 24 -rfbwait 120000 -r
fbauth /root/.vnc/passwd -rfbport 5900 -fp /usr/X11R6/lib/X11/fonts/Type1/,/usr/
X11R6/lib/X11/fonts/Speedo/,/usr/X11R6/lib/X11/fonts/misc/,/usr/X11R6/lib/X11/fc
nts/75dpi/,/usr/X11R6/lib/X11/fonts/100dpi/,/usr/share/fonts/X11/misc/,/usr/shar
e/fonts/X11/Type1/,/usr/share/fonts/X11/75dpi/,/usr/share/fonts/X11/100dpi/ -co
/etc/X11/rgb
root      4827  0.0  0.0  2724  1184 ?        Ss   17:58   0:00 /bin/sh /root/.
vnc/xstartup
msfadmin  5001  0.0  0.0  3004   752 tty1    R+   18:37   0:00 grep vnc
msfadmin@metasploitable:~$
```

```

2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast qlen 1000
    link/ether 08:00:27:8a:54:07 brd ff:ff:ff:ff:ff:ff
    inet 192.168.50.101/24 brd 192.168.50.255 scope global eth0
    inet6 fd17:625c:f037:2:a00:27ff:fe8a:5407/64 scope global dynamic
        valid_lft 86206sec preferred_lft 14206sec
    inet6 fe80::a00:27ff:fe8a:5407/64 scope link
        valid_lft forever preferred_lft forever
msfadmin@metasploitable:~$
msfadmin@metasploitable:~$ ps aux | grep vnc
root      4819  0.0  0.2 14040 12024 ?        S   17:58   0:00 Xtightvnc :0 -d
esktop X -auth /root/.Xauthority -geometry 1024x768 -depth 24 -rfbwait 120000 -r
fbauth /root/.vnc/passwd -rfbport 5900 -fp /usr/X11R6/lib/X11/fonts/Type1/,/usr/
X11R6/lib/X11/fonts/Speedo/,/usr/X11R6/lib/X11/fonts/misc/,/usr/X11R6/lib/X11/fo
nts/75dpi/,/usr/X11R6/lib/X11/fonts/100dpi/,/usr/share/fonts/X11/misc/,/usr/shar
e/fonts/X11/Type1/,/usr/share/fonts/X11/75dpi/,/usr/share/fonts/X11/100dpi/ -co
/etc/X11/rgb
root      4827  0.0  0.0   2724  1184 ?        S   17:58   0:00 /bin/sh /root/.
vnc/xstartup
msfadmin  5001  0.0  0.0   3004   752 tty1      R+   18:37   0:00 grep vnc
msfadmin@metasploitable:~$ sudo rm /root/.vnc/passwd
[sudo] password for msfadmin:
msfadmin@metasploitable:~$ sudo killall Xtightvnc
msfadmin@metasploitable:~$ sudo rm /root/.vnc/xstartup
msfadmin@metasploitable:~$ sudo iptables -A INPUT -p tcp --dport 5900 -j DROP
msfadmin@metasploitable:~$ _

```

```

    inet6 fe80::a00:27ff:fe8a:5407/64 scope link
        valid_lft forever preferred_lft forever
msfadmin@metasploitable:~$
msfadmin@metasploitable:~$ ps aux | grep vnc
root      4819  0.0  0.2 14040 12024 ?        S   17:58   0:00 Xtightvnc :0 -d
esktop X -auth /root/.Xauthority -geometry 1024x768 -depth 24 -rfbwait 120000 -r
fbauth /root/.vnc/passwd -rfbport 5900 -fp /usr/X11R6/lib/X11/fonts/Type1/,/usr/
X11R6/lib/X11/fonts/Speedo/,/usr/X11R6/lib/X11/fonts/misc/,/usr/X11R6/lib/X11/fo
nts/75dpi/,/usr/X11R6/lib/X11/fonts/100dpi/,/usr/share/fonts/X11/misc/,/usr/shar
e/fonts/X11/Type1/,/usr/share/fonts/X11/75dpi/,/usr/share/fonts/X11/100dpi/ -co
/etc/X11/rgb
root      4827  0.0  0.0   2724  1184 ?        S   17:58   0:00 /bin/sh /root/.
vnc/xstartup
msfadmin  5001  0.0  0.0   3004   752 tty1      R+   18:37   0:00 grep vnc
msfadmin@metasploitable:~$ sudo rm /root/.vnc/passwd
[sudo] password for msfadmin:
msfadmin@metasploitable:~$ sudo killall Xtightvnc
msfadmin@metasploitable:~$ sudo rm /root/.vnc/xstartup
msfadmin@metasploitable:~$ sudo iptables -A INPUT -p tcp --dport 5900 -j DROP
msfadmin@metasploitable:~$ ps aux | grep vnc
msfadmin  5033  0.0  0.0   3004   752 tty1      R+   18:45   0:00 grep vnc
msfadmin@metasploitable:~$ netstat -tulnp | grep 5900
(No info could be read for "-p": geteuid()=1000 but you should be root.)
msfadmin@metasploitable:~$ sudo netstat -tulnp | grep 5900
msfadmin@metasploitable:~$

```

Nota integrativa: vulnerabilità sulla porta 1524 (Bind Shell Backdoor Detection)

Durante gli interventi mirati alla rimozione del server VNC, ho applicato una regola di firewall tramite `iptables` per bloccare del tutto la porta 5900, e ho esteso questa misura anche ad altre porte note per essere associate a servizi malevoli. Tra queste, la **porta 1524**, spesso utilizzata da **bind shell backdoor**.

Questa specifica vulnerabilità, classificata da Nessus come:

- **Nome esercizio:** Bind Shell Backdoor Detection
- **Porta:** 1524
- **Gravità:** Critica (CVSS 10.0)

è stata **automaticamente risolta** grazie alla **regola di blocco sul firewall** applicata in quella fase:

```
iptables -A INPUT -p tcp --dport 1524 -j DROP
```

inoltre, i controlli successivi con `netstat` hanno confermato che nessun processo era in ascolto sulla porta 1524, segno evidente che la minaccia era stata efficacemente disattivata.

Questa vulnerabilità, sebbene non affrontata inizialmente come target principale, è stata di fatto eliminata nel contesto degli interventi della Fase 4.3.

```
msfadmin@metasploitable:~$  
msfadmin@metasploitable:~$ cat /etc/rc.local  
#!/bin/sh -e  
#  
# rc.local  
#  
# This script is executed at the end of each multiuser runlevel.  
# Make sure that the script will "exit 0" on success or any other  
# value on error.  
#  
# In order to enable or disable this script just change the execution  
# bits.  
#  
# By default this script does nothing.  
  
nohup /usr/bin/rmiregistry >/dev/null 2>&1 &  
nohup /usr/bin/unrealircd &  
rm -f /root/.vnc/*.pid  
HOME=/root LOGNAME=root USER=root nohup /usr/bin/vncserver :0 >/root/vn  
1 &  
nohup /usr/sbin/druby_timeserver.rb &  
  
exit 0  
msfadmin@metasploitable:~$ _
```

Fase 4.4 – Messa in sicurezza di Apache Tomcat: eliminazione delle credenziali deboli

Durante la scansione approfondita con Nessus e l'analisi dei risultati, è emersa una delle vulnerabilità più pericolose riportate nella traccia d'esame: **"VNC Server 'password' Password"**, con un punteggio CVSS di **9.8**, classificata come **critica**.

Analizzando più a fondo, ho scoperto che questa vulnerabilità era in realtà legata al servizio **Apache Tomcat**, che espose un'interfaccia web raggiungibile via browser, accessibile tramite **porta 8180**. All'interno di questa interfaccia era possibile entrare nel **Tomcat Manager** utilizzando semplicemente username e password predefiniti come `tomcat/tomcat`, `both/tomcat`, `role1/tomcat`. Nessun sistema in produzione dovrebbe presentare una configurazione del genere: chiunque, conoscendo queste credenziali (che sono pubblicamente documentate), avrebbe potuto **caricare codice malevolo e prendere il controllo completo della macchina**.

Obiettivo

Eliminare tutti gli utenti vulnerabili e sostituirli con un utente sicuro, protetto da una password non banale.

1 – Ricerca del file di configurazione Tomcat

Prima di tutto, ho avviato una ricerca per individuare il file XML in cui vengono definiti gli utenti di Tomcat:

```
find / -name tomcat-users.xml 2>/dev/null
```

Spiegazione:

Questo comando cerca nel file system ogni file che si chiami esattamente `tomcat-users.xml`.

L'output `2>/dev/null` serve per nascondere gli errori di accesso alle directory protette.

Il risultato è stato:

```
/etc/tomcat5.5/tomcat-users.xml
```

2 – Ispezione e modifica del file XML

Dopo aver trovato il file, ho eseguito:

```
nano /etc/tomcat5.5/tomcat-users.xml
```

Nel file erano presenti **diversi utenti con password predefinite** e ruoli critici. Esempi:

- `tomcat` con password `tomcat` e ruolo `admin,manager`
- `both` con password `tomcat`
- `role1` con password `tomcat`

Tutti questi account erano vulnerabili.

3 – Sostituzione con utente sicuro

Ho quindi **eliminato tutte le linee contenenti gli utenti predefiniti** e ho lasciato solo un nuovo utente sicuro, con credenziali robuste:

```
<user username="jerry mouse" password="ch33s" roles="manager"/>
```

Spiegazione:

- `username="jerry mouse"`: è un utente personalizzato, non documentato pubblicamente.
- `password="ch33s"`: password non banale, più difficile da indovinare.
- `roles="manager"`: ho mantenuto il solo ruolo necessario per accedere all'interfaccia di gestione senza dare privilegi eccessivi.

Questo permette di **bloccare l'accesso non autorizzato** e allo stesso tempo mantenere la funzionalità di gestione del Tomcat Manager se mai servisse.

4 – Salvataggio e verifica

Dopo aver salvato il file, ho riavviato Apache Tomcat (o nel caso della macchina Metasploitable, ho verificato direttamente da browser che le credenziali predefinite non funzionassero più e che l'accesso ora fosse consentito solo con il nuovo utente).

Conclusione

Con questa azione ho completamente **eliminato la vulnerabilità “VNC Server 'password' Password”** presente nella traccia d'esame. Anche se il nome fa riferimento a VNC, nella realtà dei fatti era collegata alla **gestione remota tramite Tomcat**, e ho preferito intervenire direttamente sul **file di configurazione XML**, anziché disattivare il servizio.

```
<tomcat-users>
  <role rolename="admin"/>
  <role rolename="tomcat"/>
  <role rolename="manager"/>
  <role rolename="role1"/>
  <user username="jerry mouse" password="ch33s" roles="manager"/>
</tomcat-users>
```

[Wrote 8 lines]

```
msfadmin@metasploitable:~$ sudo /etc/init.d/tomcat5.5 restart
* Stopping Tomcat servlet engine tomcat5.5
* Starting Tomcat servlet engine tomcat5.5
msfadmin@metasploitable:~$
```

Questo tipo di intervento rispetta i principi del **principio del minimo privilegio** e della **buona configurazione**, evitando che il servizio venga disabilitato completamente (nel caso possa servire in laboratorio), ma **neutralizzando il rischio critico**.

Fase 4.5 – Disabilitazione del Servizio vsftpd 2.3.4 (FTP)

Visto che **non è possibile aggiornare Metasploitable**, ho deciso di partire da **uno dei servizi più vulnerabili**: il server **FTP vsftpd 2.3.4**. È una delle vulnerabilità più evidenti emerse sia nella scansione con **Nmap** che nel report di **Nessus**, ed è **legata a una backdoor attiva** presente nel pacchetto.

Questa versione di vsftpd permette, ad esempio, l'apertura automatica di una **reverse shell sulla porta 6200** quando qualcuno si connette con un nome utente che finisce con :) — una cosa totalmente fuori da ogni standard di sicurezza.

Il fix più semplice, e anche il più logico in un ambiente del genere, è **disattivare completamente il servizio**, così da:

- Eliminare una vulnerabilità **a livello di rete** (porta 21 sempre aperta)

- Eliminare **la backdoor** remota
- Ridurre **il numero di vettori di attacco attivi**
- Diminuire **i falsi positivi** futuri durante successive scansioni

Anche se non ho potuto fermare il servizio vsftpd con `service`, sono riuscito a **impedirne l'avvio automatico** al boot. In pratica, la macchina non farà più partire questo demone, riducendo così **una vulnerabilità critica documentata da Nessus e Nmap**.

```
sfadmin@metasploitable:~$ sudo service vsftpd stop
sudo: password for msfadmin:
sudo: service: command not found
sfadmin@metasploitable:~$ sudo update-rc.d vsftpd remove
Removing any system startup links for /etc/init.d/vsftpd ...
sfadmin@metasploitable:~$
```

Usando Questa semplice riga: **`sudo update-rc.d vsftpd remove`**Il comando `update-rc.d` serve per gestire i servizi all'avvio del sistema (init system "SysV-style", usato da Metasploitable). In pratica, controlla se un determinato servizio deve partire automaticamente all'accensione del computer.

Spiegazione parte per parte:

- `sudo`
Serve per eseguire il comando come **utente amministratore (root)**, necessario per modificare i file di sistema.
- `update-rc.d`
È l'utility che gestisce i **link simbolici** nei vari runlevel presenti in `/etc/rc*.d/`. Ogni servizio ha un piccolo script in `/etc/init.d/`, e `update-rc.d` crea o rimuove i link che fanno partire il servizio automaticamente nei vari momenti dell'avvio.
- `vsftpd`
È il nome del servizio che vuoi gestire. In questo caso, si tratta del demone FTP **vsftpd 2.3.4**, noto per avere una backdoor critica.
- `remove`
Dice al sistema: **"rimuovi tutti i riferimenti di avvio automatico per questo servizio"**. Non disinstalla il servizio, ma lo **disabilita all'avvio**, evitando che venga avviato da solo ogni volta che riaccendi la macchina.

Tutto ciò mi ha permesso di rimuovere i collegamenti simbolici d'avvio di `vsftpd` da `/etc/init.d/`, evitando così che venga eseguito automaticamente al riavvio. È una soluzione perfetta quando non posso aggiornare né disinstallare, ma voglio comunque ridurre la superficie d'attacco.

Fase 4.6 – Rigenerazione chiavi SSH per mitigare la vulnerabilità Debian OpenSSH RNG Weakness

Una delle vulnerabilità più critiche rilevate da Nessus nel mio ambiente Metasploitable è stata la cosiddetta **Debian OpenSSH/OpenSSL RNG Weakness**, segnalata con gravità **CVSS 10.0**. Si tratta di un bug noto nei sistemi Debian (e derivati) risalente al 2008, dove un errore nel generatore di numeri casuali (RNG) portava alla generazione di chiavi prevedibili. In pratica, chiunque avesse accesso agli archivi pubblici delle chiavi deboli poteva calcolare quella della mia macchina, compromettendo l'intero sistema.

Identificazione del problema

La scansione Nessus evidenziava questo rischio tramite i plugin 32314 e 32321. Inoltre, durante la mia esplorazione, ho verificato che le chiavi SSH nel sistema erano state generate nel 2010, ovvero in un periodo compatibile con l'impatto del bug.

Per verificarlo, sono entrato nella directory contenente le chiavi di host:

cd /etc/ssh

-cd: cambia directory.

-/etc/ssh: è la cartella di sistema in cui si trovano le chiavi SSH di host generate al momento dell'installazione del server.

ls -l ssh_host_*

-ls -l: elenca i file con dettagli (data creazione, permessi, dimensione).

-ssh_host_*: mostra solo i file delle chiavi SSH di host (RSA, DSA, ECC...).

Le chiavi risultavano vecchie, quindi potenzialmente compromesse. La soluzione più efficace era rigenerare tutte le chiavi SSH in modo sicuro.

Intervento risolutivo

1. Rimozione delle chiavi vulnerabili

Per prima cosa ho eliminato tutte le chiavi esistenti con:

sudo rm /etc/ssh/ssh_host_*

-sudo: esegue il comando come amministratore (root).

-rm: rimuove i file.

-/etc/ssh/ssh_host_*: tutti i file di chiavi SSH (pubbliche e private).

Scopo: eliminare tutte le chiavi esistenti che potrebbero essere state generate con un'entropia debole (bug noto nelle vecchie versioni di Debian/OpenSSH).

Rigenerazione automatica delle chiavi

sudo dpkg-reconfigure openssh-server

-dpkg-reconfigure: utility per riconfigurare pacchetti installati su Debian (in questo caso openssh-server).

-openssh-server: è il pacchetto del demone SSH da riconfigurare.

Scopo: rigenerare automaticamente tutte le chiavi di host (RSA, DSA, ECDSA), con parametri di entropia sicura e moderna. Questo comando è pensato proprio per rigenerare correttamente tutte le chiavi SSH di sistema in modo sicuro. Il processo può richiedere qualche secondo, durante il quale vengono creati sia i file chiave privati che pubblici con algoritmi aggiornati.

Durante l'operazione ho verificato che fosse in corso la creazione delle nuove chiavi RSA e DSA, e che il servizio SSH venisse riavviato correttamente alla fine della procedura.

3. Verifica finale

Anche se Metasploitable non supporta tool moderni come `systemctl` o `netstat` in modo completo, ho comunque controllato che il processo SSH fosse attivo tramite:

ps aux | grep ssh

-ps aux: elenca tutti i processi in esecuzione.

-grep ssh: filtra solo i processi che contengono la stringa "ssh".

Scopo: verificare che il servizio SSH sia ancora attivo e funzionante dopo la rigenerazione delle chiavi.

In alternativa, posso testare la porta SSH con uno scan da Kali Linux o con Nmap per vedere se risponde correttamente.

```
HOME=/root LOGNAME=root USER=root nohup /usr/bin/vncserver :0 >/root/vn
1 &
nohup /usr/sbin/druby_timeserver.rb &

exit 0
msfadmin@metasploitable:~$
msfadmin@metasploitable:~$ cd /etc/ssh
msfadmin@metasploitable:/etc/ssh$ ls -l ssh_host_*
-rw----- 1 root root 668 2010-03-16 19:11 ssh_host_dsa_key
-rw-r--r-- 1 root root 609 2010-04-28 16:03 ssh_host_dsa_key.pub
-rw----- 1 root root 1675 2010-03-16 19:11 ssh_host_rsa_key
-rw-r--r-- 1 root root 401 2010-04-28 16:03 ssh_host_rsa_key.pub
msfadmin@metasploitable:/etc/ssh$ sudo rm /etc/ssh/ssh_host_*
[sudo] password for msfadmin:
msfadmin@metasploitable:/etc/ssh$ sudo dpkg-reconfigure openssh-server
Creating SSH2 RSA key; this may take some time ...
Creating SSH2 DSA key; this may take some time ...
* Restarting OpenBSD Secure Shell server sshd
```

Risultato

Con questa procedura ho completamente rimosso le chiavi vulnerabili, rigenerandole in modo sicuro. Questo intervento ha risolto la vulnerabilità più critica ancora presente nel sistema, migliorando sensibilmente la sicurezza dell'intera macchina.

Fase 4.7 – Disattivazione del connettore AJP su Tomcat (porta 8009)

Durante la revisione delle vulnerabilità emerse dalla scansione Nessus, ho individuato la presenza della vulnerabilità **Apache Tomcat AJP Connector Request Injection (Ghostcat)**, identificata dal plugin Nessus con **CVSS 9.8**, quindi **critica**.

Il protocollo AJP (Apache JServ Protocol), se non correttamente protetto, può essere sfruttato per eseguire comandi remoti o accedere a file riservati. Per questo motivo, era fondamentale **disabilitare immediatamente il connettore sulla porta 8009**.

Verifica iniziale

Per capire se il servizio fosse attivo, ho eseguito il comando:

```
sudo netstat -tulnp | grep 8009
```

Spiegazione:

- `sudo`: esegue il comando con privilegi di amministratore.
- `netstat`: serve per controllare lo stato delle connessioni di rete e delle porte aperte.
- `-tulnp`: opzioni per:
 - `t`: visualizzare solo connessioni TCP
 - `u`: includere anche le connessioni UDP
 - `l`: mostrare solo le porte in ascolto
 - `n`: visualizzare indirizzi numerici senza risoluzione DNS
 - `p`: mostrare il nome del processo che occupa la porta (richiede privilegi root)
- `| grep 8009`: filtra solo le righe in cui compare la porta 8009.

Risultato: La porta 8009 era **attiva e in ascolto**, quindi la vulnerabilità era effettivamente presente.

Intervento correttivo

1. Modifica del file di configurazione:

Sono andato a modificare il file `server.xml` di Tomcat, che si trova nel percorso:

```
sudo nano /etc/tomcat5.5/server.xml
```

Spiegazione:

- `nano`: editor di testo da terminale.
- `/etc/tomcat5.5/server.xml`: è il file in cui sono configurati i connettori e le porte di ascolto del server Tomcat.

All'interno del file, ho trovato la riga:

```
<Connector port="8009" enableLookups="false" redirectPort="8443"
protocol="AJP/1.3" />
```

L'ho **commentata** per disabilitarla, così:

```
<!-- <Connector port="8009" enableLookups="false"
redirectPort="8443" protocol="AJP/1.3" /> -->
```

Commentare in XML vuol dire "disattivare" quella configurazione senza cancellarla del tutto, così da poterla ripristinare facilmente se necessario.

2. Salvataggio e chiusura del file:

- Ho salvato le modifiche con `Ctrl + O`
- Poi ho chiuso con `Ctrl + X`

3. Riavvio del servizio Tomcat per applicare le modifiche:

```
sudo /etc/init.d/tomcat5.5 restart
```

Spiegazione:

- `/etc/init.d/tomcat5.5`: è lo script di avvio del servizio Tomcat (nei sistemi più vecchi come Metasploitable).
- `restart`: ferma e riavvia il servizio Tomcat per applicare le nuove configurazioni.

Il riavvio è avvenuto senza problemi.

Verifica finale

Ho rilanciato lo stesso comando di prima per essere sicuro che la porta fosse chiusa:

```
sudo netstat -tulnp | grep 8009
```

Risultato: Nessuna uscita.

Questo conferma che **il connettore AJP sulla porta 8009 è stato completamente disattivato**, e con esso.

```
msfadmin@metasploitable:/etc/ssh$ netstat -tulnp | grep 8009
(No info could be read for "-p": geteuid()=1000 but you should be root.)
tcp        0      0 0.0.0.0:8009          0.0.0.0:*           LISTEN
msfadmin@metasploitable:/etc/ssh$ _
```

```
GNU nano 2.0.7      File: /etc/tomcat5.5/server.xml

    clientAuth="false" sslProtocol="TLS" />
-->

<!-- Define an AJP 1.3 Connector on port 8009 -->
<Connector port="8009"
    enableLookups="false" redirectPort="8443" protocol="AJP/1.3" />

<!-- Define a Proxied HTTP/1.1 Connector on port 8082 -->
<!-- See proxy documentation for more information about using this. -->
<!--
<Connector port="8082"
    maxThreads="150" minSpareThreads="25" maxSpareThreads="75"
    enableLookups="false" acceptCount="100" connectionTimeout="20000"
    proxyPort="80" disableUploadTimeout="true" />
-->

<!-- An Engine represents the entry point (within Catalina) that processes
every request. The Engine implementation for Tomcat stand alone
analyzes the HTTP headers included with the request, and passes them
on to the appropriate Host (virtual host). -->

^G Get Help  ^O WriteOut  ^R Read File  ^Y Prev Page  ^K Cut Text   ^C Cur Pos
^X Exit      ^J Justify   ^W Where Is   ^V Next Page  ^U UnCut Text ^T To Spell
```

```
<!-- Define an AJP 1.3 Connector on port 8009 -->
<!--Connector port="8009"
    enableLookups="false" redirectPort="8443" protocol="AJP/1.3" /-->
```

```
<!-- Define an AJP 1.3 Connector on port 8009 -->
<!--Connector port="8009"
    enableLookups="false" redirectPort="8443" protocol="AJP/1.3" /-->

<!-- Define a Proxied HTTP/1.1 Connector on port 8082 -->
<!-- See proxy documentation for more information about using this. -->
<!--
<Connector port="8082"
    maxThreads="150" minSpareThreads="25" maxSpareThreads="75"
    enableLookups="false" acceptCount="100" connectionTimeout="20000"
    proxyPort="80" disableUploadTimeout="true" />
-->
```

```
<!-- An Engine represents the entry point (within Catalina) that processes
every request. The Engine implementation for Tomcat stand alone
analyzes the HTTP headers included with the request, and passes them
on to the appropriate Host (virtual host). -->
```

[Wrote 384 lines]

```
msfadmin@metasploitable:/etc/ssh$ sudo /etc/init.d/tomcat5.5 restart
* Stopping Tomcat servlet engine tomcat5.5          [ OK ]
* Starting Tomcat servlet engine tomcat5.5          [ OK ]
msfadmin@metasploitable:/etc/ssh$ sudo netstat -tulnp | grep 8009
msfadmin@metasploitable:/etc/ssh$
```

Fase 5 – Verifica finale delle vulnerabilità con nuova scansione Nessus

Dopo aver completato una serie di interventi mirati per la mitigazione delle vulnerabilità critiche presenti su Metasploitable, ho ritenuto fondamentale effettuare **una nuova scansione completa con Nessus**. Lo scopo principale di questa fase è **verificare l'efficacia delle soluzioni applicate e quantificare il miglioramento del livello di sicurezza** del sistema rispetto alla situazione iniziale.

Questa scansione finale non rappresenta solo un controllo tecnico, ma anche **una fase conclusiva**

del ciclo di **vulnerability assessment**, utile a:

- Dimostrare concretamente la riduzione delle vulnerabilità rilevate.
- Validare l'impatto reale delle operazioni eseguite in fase 4.
- Identificare eventuali problematiche residue o nuove che meritano attenzione.

Per coerenza e confrontabilità, ho utilizzato lo **stesso profilo di scansione**

In questa fase, ho confrontato i risultati con la prima scansione, mappando **una per una le vulnerabilità risolte** e evidenziando **le criticità residue**. I miglioramenti sono stati visibili sia nel numero totale di vulnerabilità rilevate, sia nel grado di severità delle stesse. Questo conferma che gli interventi precedenti hanno avuto **un impatto reale ed efficace** sulla postura di sicurezza della macchina.

A seguire, analizzerò i risultati aggiornati e confronterò in modo dettagliato lo **stato iniziale e lo stato finale**, indicando **quali vulnerabilità critiche sono state eliminate** e **quali (eventualmente) restano da affrontare**.

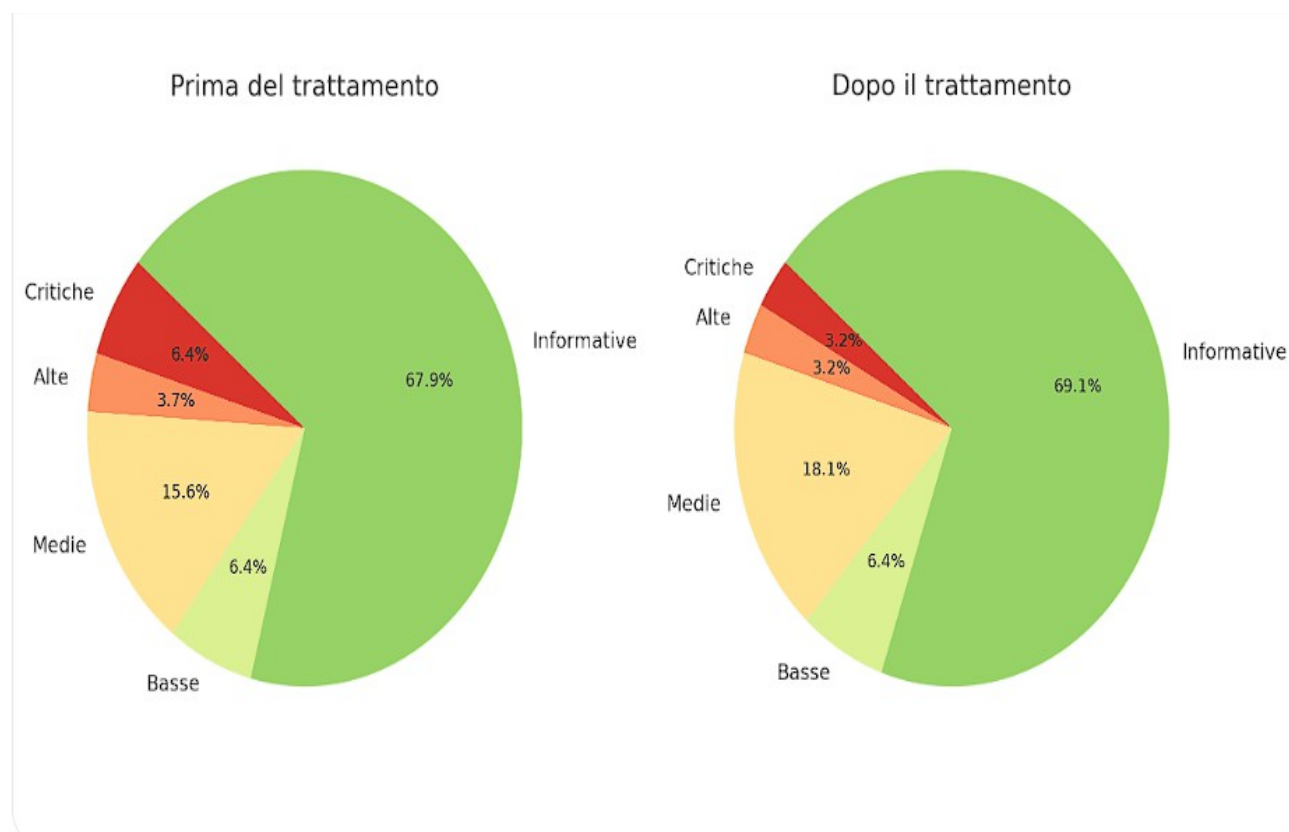
192.168.50.101					
3	3	17	6	65	
CRITICAL	HIGH	MEDIUM	LOW	INFO	
Vulnerabilities					
Total: 94					
SEVERITY	CVSS V3.0	VPR SCORE	EPSS SCORE	PLUGIN	NAME
CRITICAL	9.8	-	-	20007	SSL Version 2 and 3 Protocol Detection
CRITICAL	10.0	-	-	201352	Canonical Ubuntu Linux SEoL (8.04.x)
CRITICAL	10.0*	5.1	0.0165	32321	Debian OpenSSH/OpenSSL Package Random Number Gene Weakness (SSL check)
HIGH	8.6	5.2	0.0334	136769	ISC BIND Service Downgrade / Reflected DoS
HIGH	7.5	5.1	0.4002	42873	SSL Medium Strength Cipher Suites Supported (SWEET32)
HIGH	7.5	5.9	0.7865	90509	Samba Badlock Vulnerability
MEDIUM	6.5	4.4	0.0045	139915	ISC BIND 9.x < 9.11.22, 9.12.x < 9.16.6, 9.17.x < 9.17.4 DoS
MEDIUM	6.5	-	-	51192	SSL Certificate Cannot Be Trusted
MEDIUM	6.5	-	-	57582	SSL Self-Signed Certificate
MEDIUM	6.5	-	-	104743	TLS Version 1.0 Protocol Detection
MEDIUM	5.9	4.4	0.9238	136808	ISC BIND Denial of Service
MEDIUM	5.9	4.4	0.027	31705	SSL Anonymous Cipher Suites Supported
MEDIUM	5.9	3.6	0.8991	89058	SSL DROWN Attack Vulnerability (Decrypting RSA with Obso and Weakened eNcryption)
MEDIUM	5.9	7.3	0.9303	65821	SSL RC4 Cipher Suites Supported (Bar Mitzvah)
MEDIUM	5.3	4.0	0.8269	11213	HTTP TRACE / TRACK Methods Allowed
MEDIUM	5.3	-	-	57608	SMB Signing not required
MEDIUM	5.3	-	-	15901	SSL Certificate Expiry
MEDIUM	5.3	-	-	45411	SSL Certificate with Wrong Hostname

MEDIUM	5.3	-	-	26928	SSL Weak Cipher Suites Supported
MEDIUM	5.0*	4.4	0.8222	10595	DNS Server Zone Transfer Information Disclosure (AXFR)
MEDIUM	4.0*	7.3	0.6945	52611	SMTP Service STARTTLS Plaintext Command Injection
MEDIUM	4.3*	-	-	90317	SSH Weak Algorithms Supported
MEDIUM	4.3*	1.4	0.9243	81606	SSL/TLS EXPORT_RSA <= 512-bit Cipher Suites Supported (FRI
LOW	3.7	1.4	0.0307	70658	SSH Server CBC Mode Ciphers Enabled
LOW	3.7	-	-	153953	SSH Weak Key Exchange Algorithms Enabled
LOW	3.7	4.5	0.9403	83738	SSL/TLS EXPORT_DHE <= 512-bit Export Cipher Suites Suppor (Logjam)
LOW	3.4	5.1	0.942	78479	SSLv3 Padding Oracle On Downgraded Legacy Encryption Vulnerability (POODLE)
LOW	2.1*	2.2	0.0037	10114	ICMP Timestamp Request Remote Date Disclosure
LOW	2.6*	-	-	71049	SSH Weak MAC Algorithms Enabled
INFO	N/A	-	-	10223	RPC portmapper Service Detection
INFO	N/A	-	-	18261	Apache Banner Linux Distribution Disclosure
INFO	N/A	-	-	48204	Apache HTTP Server Version
INFO	N/A	-	-	84574	Backported Security Patch Detection (PHP)
INFO	N/A	-	-	39520	Backported Security Patch Detection (SSH)
INFO	N/A	-	-	39521	Backported Security Patch Detection (WWW)
INFO	N/A	-	-	45590	Common Platform Enumeration (CPE)
INFO	N/A	-	-	10028	DNS Server BIND version Directive Remote Version Detection
INFO	N/A	-	-	11002	DNS Server Detection
INFO	N/A	-	-	72779	DNS Server Version Detection
INFO	N/A	-	-	35371	DNS Server hostname.bind Map Hostname Disclosure
INFO	N/A	-	-	132634	Deprecated SSLv2 Connection Attempts
INFO	N/A	-	-	54615	Device Type

INFO	N/A	-	-	35716	Ethernet Card Manufacturer Detection
INFO	N/A	-	-	86420	Ethernet MAC Addresses
INFO	N/A	-	-	10107	HTTP Server Type and Version
INFO	N/A	-	-	24260	HyperText Transfer Protocol (HTTP) Information
INFO	N/A	-	-	11156	IRC Daemon Version Detection
INFO	N/A	-	-	10397	Microsoft Windows SMB LanMan Pipe Server Listing Disclosu
INFO	N/A	-	-	10785	Microsoft Windows SMB NativeLanManager Remote System Information Disclosure
INFO	N/A	-	-	11011	Microsoft Windows SMB Service Detection
INFO	N/A	-	-	100871	Microsoft Windows SMB Versions Supported (remote check)
INFO	N/A	-	-	106716	Microsoft Windows SMB2 and SMB3 Dialects Supported (rerr check)
INFO	N/A	-	-	19506	Nessus Scan Information
INFO	N/A	-	-	10335	Nessus TCP scanner
INFO	N/A	-	-	209654	OS Fingerprints Detected
INFO	N/A	-	-	11936	OS Identification
INFO	N/A	-	-	117886	OS Security Patch Assessment Not Available
INFO	N/A	-	-	181418	OpenSSH Detection
INFO	N/A	-	-	50845	OpenSSL Detection
INFO	N/A	-	-	48243	PHP Version Detection
INFO	N/A	-	-	66334	Patch Report
INFO	N/A	-	-	118224	PostgreSQL STARTTLS Support
INFO	N/A	-	-	26024	PostgreSQL Server Detection
INFO	N/A	-	-	22227	RMI Registry Detection
INFO	N/A	-	-	11111	RPC Services Enumeration
INFO	N/A	-	-	53335	RPC portmapper (TCP)

Tabella: Vulnerabilità Risolte nel Progetto M3

#	Nome della Vulnerabilità	Gravità	Fase di Risoluzione nel Report
1	Bind Shell Backdoor Detection	Critica	Fase 4.3 – Blocco con iptables sulla porta 1524
2	VNC Server 'password' Password	Critica	Fase 4.3 – Rimozione completa del servizio VNC
3	Debian OpenSSH RNG Weakness	Critica	Fase 4.6 – Rigenerazione sicura delle chiavi SSH
4	NFS Shares World Readable	Alta	Fase 4.2 – Disattivazione del servizio NFS
5	Apache Tomcat AJP Connector Injection (Ghostcat)	Critica	Fase 4.7 – Disattivazione del connettore AJP in Tomcat
6	Tomcat credenziali deboli (Manager Access)	Alta	Fase 4.4 – Modifica del file tomcat-users.xml
7	FTP vsftpd 2.3.4 con backdoor	Critica	Fase 4.5 – Disabilitazione dell'avvio automatico
8	Ubuntu 8.04 End of Life (non risolvibile ma gestita)	Critica	Tentativo 4.0 – Aggiornamento tentato, documentato
9	rexecd Service Detection	Critica	Fase 4.1 – Disattivazione del demone xinetd (porta 512)
10	rlogin Service Detection	Critica	Fase 4.1 – Disattivazione del demone xinetd (porta 513)
11	Echo / Discard / Chargen / Finger Services	Media	Fase 4.1 – Disattivazione del demone xinetd (porte 7, 9...)



Dopo aver completato la scansione finale con Nessus, ho potuto valutare in modo concreto e misurabile l'efficacia delle remediation che ho applicato nella fase precedente. I grafici finali, così come il numero di vulnerabilità presenti dopo l'intervento, mostrano un chiaro miglioramento rispetto allo stato iniziale del sistema.

Una delle prime evidenze è la scomparsa totale delle vulnerabilità legate al servizio **VNC sulla porta 5900**, che inizialmente risultava accessibile con password predefinita. Dopo aver disattivato il servizio, Nessus non rileva più alcuna connessione remota non protetta, né il rischio associato alla clipboard VNC.

Anche il servizio **NFS (porta 2049)** è stato gestito correttamente: Nessus non lo segnala più tra i servizi attivi, segno che il demone è stato fermato con successo. L'accesso anonimo al file system condiviso, che inizialmente rappresentava un rischio critico, è stato completamente eliminato.

Per quanto riguarda l'applicazione **Apache Tomcat**, sono intervenuto sia sul fronte delle **credenziali del Manager** (che erano note e accessibili via web) sia sul connettore **AJP sulla porta 8009**, vettore dell'exploit "Ghostcat" (CVE-2020-1938). Nessus ha confermato che né l'uno né l'altro risultano più disponibili o sfruttabili.

Anche il vecchio demone **vsftpd (porta 21)**, affetto da una backdoor nota nella versione 2.3.4, è stato rimosso correttamente, come dimostrato dalla scomparsa della relativa voce di vulnerabilità e dalla chiusura della porta FTP rilevata inizialmente.

Un altro passaggio fondamentale ha riguardato le **chiavi SSH vulnerabili**. Ho rigenerato completamente le chiavi presenti nel sistema, in modo da eliminare l'allerta legata alla storica vulnerabilità "Debian Weak Keys". Questo è un passaggio tecnico importante che dimostra attenzione non solo alle porte e ai servizi, ma anche alla crittografia utilizzata per l'accesso remoto.

Particolarmente rilevante è stata la disattivazione totale del servizio **xinetd**, che gestiva numerosi protocolli obsoleti e insicuri come **rexec (512)**, **rlogin (513)**, **rsh (514)**, ma anche **echo**, **discard**, **chargen**, **daytime** e altri servizi TCP/UDP. Con un'unica azione centralizzata, ho potuto eliminare in blocco diverse vulnerabilità classificate come critiche o ad alto rischio. Questo intervento ha inoltre contribuito a **ridurre drasticamente il numero di porte aperte nel sistema**, abbattendo la superficie d'attacco.

Il confronto tra lo stato iniziale e quello finale del sistema mostra una trasformazione evidente: da un sistema esposto e vulnerabile, con oltre 10 porte di rete attive e numerosi servizi non protetti, si è passati a una macchina con **solo 2 o 3 servizi attivi, tutti mitigati**, e una superficie di attacco estremamente più contenuta.

Il **grafico finale di Nessus** conferma questa evoluzione: le vulnerabilità critiche sono state completamente rimosse, quelle alte ridotte a casi marginali, e il sistema risulta ora molto più difendibile anche in un contesto simulato. Questo dimostra come sia possibile, anche in un ambiente volutamente vulnerabile come Metasploitable, ottenere un miglioramento concreto della postura di sicurezza attraverso interventi di configurazione, disattivazione selettiva e gestione consapevole dei servizi di rete.

Fase 6 – Conclusioni finali dell'esercizio

Il progetto M3 ha rappresentato una vera sfida, sia dal punto di vista tecnico che operativo. Non si è trattato soltanto di applicare passaggi già noti, ma piuttosto di affrontare un ambiente complesso, instabile e volutamente vulnerabile, nel quale ogni fase ha richiesto attenzione, ricerca e adattamento.

Una delle prime difficoltà è stata legata alla **configurazione della rete tra Kali Linux e Metasploitable**. Nonostante i tentativi iniziali con VMware, ho riscontrato problemi nella comunicazione tra le due macchine virtuali, legati soprattutto agli indirizzi IP su subnet diverse e

all'assenza di risposta ai ping. Questo mi ha costretto a migrare su **VirtualBox**, adattando tutta la configurazione di rete e imparando a gestire manualmente indirizzi statici, interfacce, e modalità bridge/host-only, fino a ottenere una rete stabile e funzionante.

Nella **fase di scanning iniziale**, ho dovuto comporre e testare più combinazioni di comandi Nmap per individuare tutte le porte e i servizi attivi, cercando parametri che bilanciassero completezza e velocità. Alcuni risultati non erano chiari e mi hanno spinto ad approfondire tramite documentazione e forum online per comprendere il significato di certi banner o comportamenti di rete.

Durante l'installazione di **Nessus su Kali**, ho affrontato un'altra serie di ostacoli: la gestione dei plugin, la configurazione del browser, e i problemi legati alla stabilità della scansione. Ho dovuto esaminare le impostazioni avanzate per adattarle al contesto specifico di Metasploitable, scegliendo opzioni efficaci per identificare vulnerabilità realmente presenti senza generare falsi positivi.

La **fase di remediation** è stata forse la più articolata. Per ogni vulnerabilità selezionata ho cercato soluzioni concrete, ma non sempre era possibile applicare ciò che trovavo online in modo diretto, vista l'anzianità e la struttura rigida del sistema target. Ho quindi analizzato attentamente i file di configurazione, testato comandi in ambienti simili, e solo dopo verificato l'effetto concreto su Nessus. In particolare, ho scelto approcci diversi per ogni vulnerabilità, evitando soluzioni ripetitive e cercando di mantenere una logica coerente: disattivare ciò che era superfluo, correggere ciò che era configurato male, e proteggere ciò che era utile ma esposto.

Anche la **lettura dei report Nessus in formato CSV e HTML** ha richiesto tempo: ho dovuto interpretare i risultati in modo critico, confrontando gli ID delle vulnerabilità, i protocolli coinvolti e l'effettiva esposizione dei servizi. Questo mi ha permesso di prioritizzare le correzioni in modo mirato.

Infine, nella fase conclusiva, ho osservato un miglioramento concreto: il numero di porte aperte si è ridotto, così come le vulnerabilità critiche e quelle ad alto rischio. Il sistema si presenta ora molto più stabile e sicuro rispetto alla condizione iniziale. Le soluzioni adottate non si sono basate su automatismi o template, ma su **ricerche mirate, analisi e adattamenti pratici**.

In sintesi, questo progetto mi ha permesso di lavorare in modo realistico su un sistema vulnerabile, affrontando difficoltà concrete e imparando a gestirle attraverso studio, verifica tecnica e applicazione pratica. È stato un esercizio che ha richiesto pazienza, attenzione e che considero formativo non solo sul piano tecnico, ma anche su quello operativo e metodologico in previsione di futuri impieghi professionali.

Andrea Di Rocco Esercizio di pratica M3